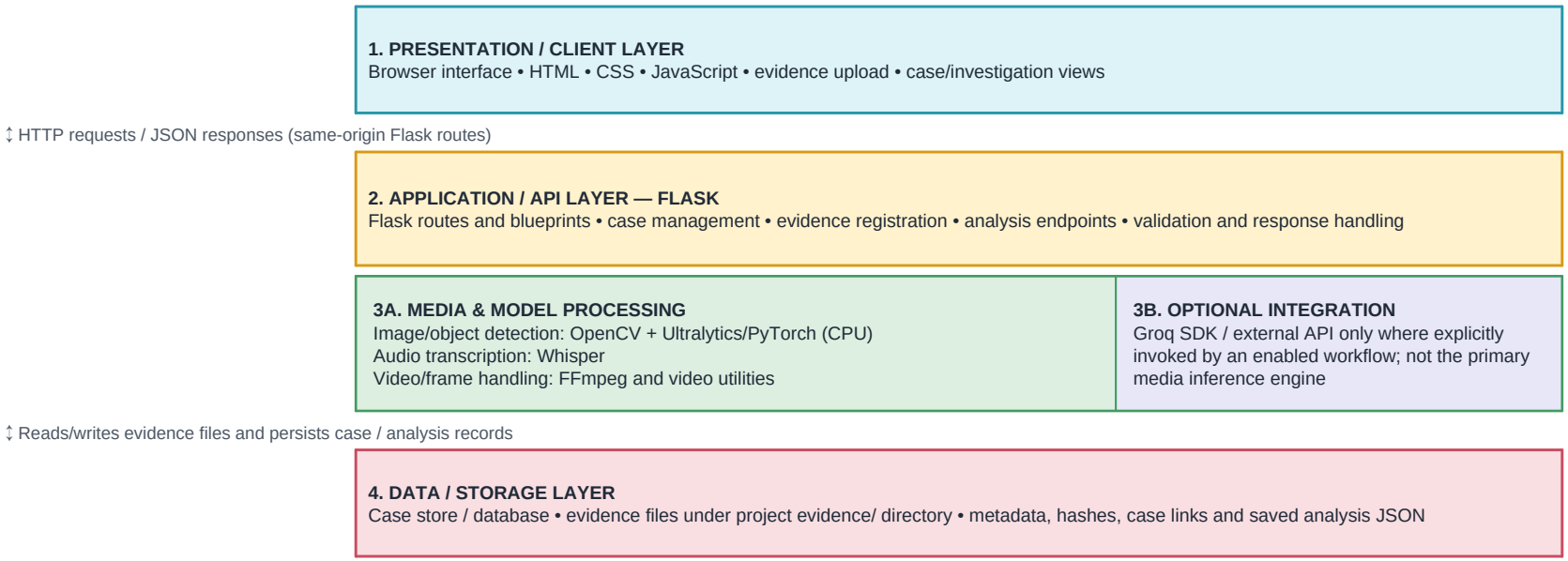
# SOLUTION ARCHITECTURE

CrimeVision — Intelligent Multimodal Forensic Analysis Platform

| Date | Team ID | Project Name | Architecture |
|---|---|---|---|
| 1 November 2026 | [Enter Team ID] | CrimeVision | Local prototype / layered web application |

## Solution Architecture Diagram




Deployment note: The current development setup runs in WSL 2 Ubuntu using a Python virtual environment. PyTorch is CPU-only; no CUDA GPU is required for the current prototype. This is a layered Flask application, not a microservices deployment, and the diagram does not imply a separate API gateway or authentication service unless those are added.

## Component Description Table

| Component Name | Description / Role in Architecture | Technologies Used |
|---|---|---|
| Presentation / Client Layer | Allows users to interact with cases, upload evidence, browse registered files, and view analysis results. | HTML, CSS, JavaScript; browser |
| Application / API Layer | Receives HTTP requests, validates inputs, coordinates case/evidence operations, invokes supported analysis workflows, and returns JSON or files. | Python, Flask, Flask-CORS |
| Media & Model Processing | Runs supported image/object-detection and audio/video processing tasks. Model output is assistance for human review, not a forensic conclusion. | OpenCV, Ultralytics, PyTorch CPU, Whisper, FFmpeg |
| Data / Storage Layer | Stores case/evidence/analysis metadata and JSON results; keeps uploaded originals in local evidence storage and records file hashes where provided by the workflow. | Project case store/database; local filesystem; JSON |

| Component Name | Description / Role in Architecture | Technologies Used |
|---|---|---|
| Optional External Integration | May call an external service for a specific enabled feature; not required for local image inference and should only be described as active if the app calls it. | Groq SDK / API, when configured and used |
| Security / Trust Boundary | Current prototype requires further hardening before real sensitive case use, including authentication/authorization, audit logging, secure deployment, and validated chain-of-custody controls. | Application-level controls to be implemented/verified |

Important: Replace [Enter Team ID] before submission. Keep the architecture aligned with the code you actually demonstrate; features marked optional or requiring implementation should not be claimed as completed.